

# Loan Repayment Service

## for MSME Lending

Build a Next.js application that generates a loan repayment schedule, records payments against that schedule, and reports the current position of the loan at any time. Assessment covers the correctness of the money arithmetic, the quality of a small test suite, and whether the repository runs as documented without intervention.

| DEADLINE                           | EXPECTED EFFORT | STACK                                         | SUBMIT TO                        |
|------------------------------------|-----------------|-----------------------------------------------|----------------------------------|
| <b>Fri 18 Sept<br/>4:00 PM IST</b> | <b>~5 hours</b> | <b>Next.js · PostgreSQL<br/>Firebase Auth</b> | <b>sourav.shukla@vitto.money</b> |

### 00 CONTEXT

Vitto does not lend money. Vitto builds the software that banks, NBFCs and microfinance institutions use to operate lending end-to-end. This exercise covers a narrow slice of the repayment side of that system.

Loans range from ₹50,000 to ₹10,00,000 over three to thirty-six months and are repaid monthly. Payments frequently arrive for an incorrect amount, after the due date, or split across more than one transaction. The service must remain correct under those conditions.

### 01 REQUIREMENTS

#### API Three REST endpoints

REQUIRED

Implement as Next.js route handlers. Route naming and response shapes are at your discretion but must be consistent across endpoints, including error responses.

- **Create a loan.** Accepts principal, annual interest rate, tenure in months and disbursement date. Generates and persists the full repayment schedule.
- **Get a loan.** Returns the schedule — per instalment: due date, principal component, interest component, total due, amount paid — and the current position: outstanding principal, next due date and amount, and any overdue amount.
- **Record a payment.** Accepts an amount and a date. Allocates the amount across the schedule.

#### DB PostgreSQL

REQUIRED

- Local PostgreSQL or a hosted instance (Neon, Supabase or equivalent).
- Schema must be created by a script or on startup, not manually.
- Money must not be stored as a floating point type.
- A payment must not be able to exist without an associated loan. Enforce this in the schema, not only in application code.
- Data access may use an ORM such as Prisma, or plain SQL. This choice is not assessed.

## AUTH Firebase

REQUIRED

- Authentication via Firebase Authentication, using email and password or Google.
- All three route handlers must reject unauthenticated requests. Tokens must be verified server-side, not only in the client.
- The interface must direct unauthenticated users to sign in, and must provide a sign-out action.
- Excluded: roles, per-user data ownership, password reset, email verification. Loans are not owned by individual users.
- Firebase credentials must not be committed. Provide a `.env.example` listing the required variables, and send the actual values and a test account in the submission email.

## UI One page

REQUIRED

A single page implemented in React within the same Next.js application. Required functionality:

- Display a loan's schedule as a table, with the current position and any overdue amount visible.
- Record a payment and reflect the result without a manual page refresh.

Component libraries and styling are not required. Loan creation may be performed through the API or a seed script; a form is not required.

## TEST Tests

REQUIRED

Approximately eight to twelve tests. Tests should target behaviour that affects correctness rather than maximise count.

- **Unit tests** covering schedule generation and payment allocation, including the cases in section 02.
- **Two or three integration tests** against the route handlers and a real database rather than mocks: one success path, one failure, and one confirming that an unauthenticated request is rejected.
- The suite must run from a single documented command.

## 02 CASES TO HANDLE

The following must be handled. Where the correct behaviour is a judgement call, document the decision taken.

- **Underpayment.** The instalment is ₹9,986 and ₹5,000 is received.
- **Overpayment.** Twice the instalment is received. Determine whether this settles the following instalment or reduces principal.
- **Late payment.** Payment is received eleven days after the due date. Define how this is reflected in the loan position.
- **Duplicate submission.** The same payment submitted twice must not be applied twice.
- **Invalid input.** Negative amounts, zero-month tenure, non-numeric values, unknown loan identifier.

**Reference formula.** Equal monthly instalments:

$$EMI = P \times r \times (1 + r)^n \div ((1 + r)^n - 1)$$

where P is principal, n is tenure in months and r is the monthly rate, being the annual rate divided by twelve and by one hundred. Verification: ₹2,00,000 at 18% per annum over 24 months yields approximately **₹9,986 per month**. Rounding variance of one or two rupees is acceptable. The final instalment is the conventional place for the remainder.

**Allocation order.** The order in which a payment settles interest, principal and outstanding instalments is at your discretion. Any defensible order is acceptable. The order must be documented.

### 03 VERIFICATION STEPS

The following will be executed on a clean machine, following your setup instructions, in this order:

1. `npm install`, environment values applied, then the documented database setup step.
2. `npm run dev`.
3. The documented test command.
4. Sign in, open the page, view a loan, record a payment.

Verify these steps against a fresh clone before submitting. A repository that does not start is the most common cause of failure in this exercise.

### 04 CONSTRAINTS

- **Stack:** Next.js with React, PostgreSQL, Firebase Authentication, REST. TypeScript or JavaScript.
- **Git:** commit in discrete, meaningful steps. Do not squash the exercise into a single commit. Commit history will be reviewed.
- **Out of scope:** user roles, per-user data, multiple currencies, prepayment closure, penalty interest, deployment.
- **If time runs short:** prioritise schedule generation, payment allocation and tests. Reduce the interface.

**AI tools.** Permitted. You must be able to account for any code submitted. The interview will examine specific functions and ask how your allocation logic behaves in a case not specified in this brief.

### 05 DELIVERABLES & SUBMISSION

01

#### Git Repository

Public or shared access. Discrete commit history. Contains schema setup, tests, `.env.example` and README. No credentials committed.

02

#### README

One page, covering: setup steps, which database you used, the test command, an endpoint reference, the money type chosen, and your allocation and rounding decisions.

#### Submission

Email [sourav.shukla@vitto.money](mailto:sourav.shukla@vitto.money) with the subject line **Full Stack SDE — Assignment — [Your Name]**. Grant access if the repository is private. Include environment values and a test account in the email, not in the repository.

**Deadline: Friday 18 September, 4:00 PM IST.**